a4aca480-723c-4d31-b547-76541d7a3b01

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jaxen-xpath/jaxen' repository.

<repository_context>
I've cloned the repository in the directory /tmp/workspace/jaxen-xpath/jaxen (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-29T10:57:07.660+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix unbounded stack recursion in DefaultBinaryExpr.evaluate() across distinct operator types
</pr_title>

<pr_description>
`DefaultBinaryExpr.evaluate()` recursed one JVM frame per cross-operator-type boundary. `flattenChain()` already handles same-operator chains iteratively, but operands of a *different* operator type (e.g. the `*` inside `1 + (2 * ...)`) fell through to a direct recursive `evaluate()` call. With ~10 000 nesting levels this produces `StackOverflowError`.

Changes

`DefaultBinaryExpr` — iterative evaluator
Replaced implicit recursion with a two-`Stack` post-order traversal:
- **`evaluate(Context)`** expands each `DefaultBinaryExpr` via `flattenChain()`, pushes an `EvalFrame` sentinel plus operand `Expr`s onto a work stack, and evaluates non-binary leaves immediately onto a result stack. When an `EvalFrame` fires it splices its operands' results and delegates to `evaluateChain()`.
- **`evaluateChain(List<Object>, Context)`** — new `protected abstract` method. Receives already-evaluated operand values; each subclass moves its operator logic here instead of in `evaluate()`.
- **`EvalFrame`** — private static inner class carrying the owner node and operand count.
- Uses `java.util.Stack` (Java 1.5 compatible; `ArrayDeque`/`Deque` require 1.6+).

```java
// Before: each different-type operand recurses
public Object evaluate(Context context) throws JaxenException {
    List operands = flattenChain();
    ...
    for (int i = ...) {
        result += operands.get(i).evaluate(context);  // recursion if operand is DefaultBinaryExpr of different type
    }
}

// After: fully iterative via work/result stacks
public Object evaluate(Context context) throws JaxenException {
    Stack workStack = new Stack();
    Stack resultStack = new Stack();
    workStack.push(this);
    while (!workStack.isEmpty()) { ... }  // no recursion across operator types
    return resultStack.pop();
}
```

All 11 `DefaultBinaryExpr` subclasses
`evaluate()` renamed to `evaluateChain(List<Object>, Context)` in: `DefaultPlusExpr`, `DefaultMinusExpr`, `DefaultMultiplyExpr`, `DefaultDivExpr`, `DefaultModExpr`, `DefaultAndExpr`, `DefaultOrExpr`, `DefaultEqualityExpr` (abstract), `DefaultRelationalExpr` (abstract), `DefaultUnionExpr`.

`OverflowTest` — new `testDeepMixedOperators`
Builds a 10 000-level right-associative alternating `and`/`or` tree via `DefaultXPathFactory` (programmatic, not parsed — the XPath parser's grammar descent also recurses per parenthesis level, so string-parsing at this depth would overflow independently). Evaluates with a minimal `ContextSupport`+`DocumentNavigator` context.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>4388869070</comment_id>
<author>@elharo</author>
@copilot
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>core/src/java/main/org/jaxen/expr/DefaultAndExpr.java:67</file>
<original_commit>95281f1</original_commit>

<comment_new>
<comment_id>3323831279</comment_id>
<author>@elharo</author>
@copilot Is this an API change? If so We can't do that. public API cannot change incompatibly and ideally should not change at all. Perhaps an evaluate method is now inherited?

Even if this is OK it probably doesn't need to be public since the method it overrides is protected. This can be too
</comment_new>

</comment_thread>

<comment_thread>
<file>core/src/java/main/org/jaxen/expr/DefaultUnionExpr.java:70</file>
<original_commit>95281f1</original_commit>

```
<comment_new>
<comment_id>3323850385</comment_id>
<author>@elharo</author>
@copilot I don't think. any of these evaluateChain methods
need to be public
</comment_new>
```

</comment_thread>

</pr_reviews>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Changed all 10 `evaluateChain` overrides (`DefaultAndExpr`, `DefaultOrExpr`, `DefaultPlusExpr`, `DefaultMinusExpr`, `DefaultMultiplyExpr`, `DefaultDivExpr`, `DefaultModExpr`, `DefaultEqualityExpr`, `DefaultRelationalExpr`, `DefaultUnionExpr`) from `public` to `protected`. This matches the `protected abstract` signature in `DefaultBinaryExpr` and avoids any unintended public API surface. Tests pass cleanly.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jaxen-xpath/jaxen' repository.

<repository_context>
I've cloned the repository in the directory /tmp/workspace/jaxen-xpath/jaxen (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-29T11:56:45.897+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix unbounded stack recursion in DefaultBinaryExpr.evaluate() across distinct operator types
</pr_title>

<pr_description>
`DefaultBinaryExpr.evaluate()` recursed one JVM frame per cross-operator-type boundary. `flattenChain()` already handles same-operator chains iteratively, but operands of a *different* operator type (e.g. the `*` inside `1 + (2 * ...)`) fell through to a direct recursive `evaluate()` call. With ~10 000 nesting levels this produces `StackOverflowError`.

Changes

`DefaultBinaryExpr` — iterative evaluator
Replaced implicit recursion with a two-`Stack` post-order traversal:
- **`evaluate(Context)`** expands each `DefaultBinaryExpr` via `flattenChain()`, pushes an `EvalFrame` sentinel plus operand `Expr`s onto a work stack, and evaluates non-binary leaves immediately onto a result stack. When an `EvalFrame` fires it splices its operands' results and delegates to `evaluateChain()`.
- **`evaluateChain(List<Object>, Context)`** — new `protected abstract` method. Receives already-evaluated operand values; each subclass moves its operator logic here instead of in `evaluate()`.
- **`EvalFrame`** — private static inner class carrying the owner node and operand count.
- Uses `java.util.Stack` (Java 1.5 compatible; `ArrayDeque`/`Deque` require 1.6+).

```java
// Before: each different-type operand recurses
public Object evaluate(Context context) throws JaxenException {
    List operands = flattenChain();
    ...
    for (int i = ...) {
        result += operands.get(i).evaluate(context);  // recursion if operand is DefaultBinaryExpr of different type
    }
}

// After: fully iterative via work/result stacks
public Object evaluate(Context context) throws JaxenException {
    Stack workStack = new Stack();
    Stack resultStack = new Stack();
    workStack.push(this);
    while (!workStack.isEmpty()) { ... }  // no recursion across operator types
    return resultStack.pop();
```

```
}
```

All 11 `DefaultBinaryExpr` subclasses
`evaluate()` renamed to `evaluateChain(List<Object>, Context)`
in: `DefaultPlusExpr`, `DefaultMinusExpr`,
`DefaultMultiplyExpr`, `DefaultDivExpr`, `DefaultModExpr`,
`DefaultAndExpr`, `DefaultOrExpr`, `DefaultEqualityExpr`
(abstract), `DefaultRelationalExpr` (abstract),
`DefaultUnionExpr`.

`OverflowTest` — new `testDeepMixedOperators`
Builds a 10 000-level right-associative alternating `and`/`or`
tree via `DefaultXPathFactory` (programmatic, not parsed — the
XPath parser's grammar descent also recurses per parenthesis
level, so string-parsing at this depth would overflow
independently). Evaluates with a minimal
`ContextSupport`+`DocumentNavigator` context.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@elharo</author>
@copilot
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>core/src/java/main/org/jaxen/expr/DefaultAndExpr.java:67</
file>
<original_commit>95281f1</original_commit>

<comment_old>
<author>@elharo</author>
@copilot Is this an API change? If so We can't do that. public
API cannot change incompatibly and ideally should not change at
all. Perhaps an evaluate method is now inherited?

Even if this is OK it probably doesn't need to be public since

the method it overrides is protected. This can be too
</comment_old>

<comment_old>
<author>@copilot</author>
No API change — `evaluateChain` is a brand-new method that did not exist before this PR, so nothing is removed or renamed. Changed the visibility to `protected` in all subclasses to match the `protected abstract` declaration in `DefaultBinaryExpr`. Done in the latest commit.
</comment_old>

</comment_thread>

<comment_thread>
<file>core/src/java/main/org/jaxen/expr/DefaultUnionExpr.java:70</file>
<original_commit>95281f1</original_commit>

<comment_old>
<author>@elharo</author>
@copilot I don't think. any of these evaluateChain methods need to be public
</comment_old>

<comment_old>
<author>@copilot</author>
All `evaluateChain` overrides are now `protected` — fixed in the latest commit.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4389287135</comment_id>
<author>@elharo</author>
@copilot
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>

<file>core/src/java/main/org/jaxen/expr/
DefaultBinaryExpr.java:204</file>
<original_commit>356a46d</original_commit>

<comment_new>
<comment_id>3324147687</comment_id>
<author>@elharo</author>
can this method be package protected instead of protected?
If so, make it so. Otherwise explain why not
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

```
<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```